Planning to create features

A data engineer may build Gold tables from Silver tables for consumption by the business. At the same time, a data scientist is building features from the same Silver tables for models. If we aren't careful, two people working separately without communication can create different versions of the same metrics. When architecting your unified DI Platform, be sure to think about reusability and maintainability. For this reason, with features specifically, the features-as-code approach is our recommendation. Features-as-code refers to the software development practice *everything is code*, with a focus on creating a repository of reusable code to define features rather than features stored in tables.

You can implement features-as-code in various ways. Initially, we mainly focus on function reusability. You can place functions you execute in multiple notebooks or scripts in a folder within the repository root directory. In the *Applying our learning* section, you will see this is where we store functions even when not calculating a feature per se. We call these the utils. You will be referencing the `mlia_utils` notebook throughout the example projects.

You can find the `mlia_utils` functions in the root folder of the GitHub repository (`https://github.com/PacktPublishing/Databricks-Lakehouse-ML-In-Action`). We walk through pulling the GitHub repository into Databricks in the *Applying our learning* section. In it, you will find Python files containing useful functions we will use in the projects. It is best practice to save, share, and track functions so that the metrics and features we calculate are consistent. Note the empty `__init__.py` file is also in the `utils` folder. Having an `__init__.py` file is required. With this structure, we can use all functions as imports, for example, from `mlia_utils.rag_funcs` import `extract_doc_text`.

Features-as-code is not only a way to reduce duplicative work by reusing functions. It is also a great way to ensure consistent business metrics. For example, online advertising often has complex calculations for the different types of revenue. Therefore, if other people or teams calculate business-critical metrics differently, it will be hard to establish the true metric value. Instead, you can often avoid this confusion by providing executive-approved functions for use. We will talk about this again in *Chapters 5* and *6*. In addition to features being centrally located and thus easier to find, Databricks offers easy ways to document your data assets.

Creating a business process requiring the team to document tables and functions with descriptions makes current and previous efforts more discoverable. In the Databricks Catalog UI, you should see *AI-generated* suggestions to fill in your table and column descriptions, so you don't have to start from scratch. Another great way to document transformations performed on a data table would be using tags. Tags can help with documentation and communication. Recall the example of missing data being imputed (*Figure 2.5*).

Focusing on ML features more specifically, you will learn how to store and serve feature functions to simplify your final deployment process. You will create an on-demand feature function and use it in your model. We also will show you how to leverage saved feature tables to create training sets. If you want to jump ahead right away, see *Chapter 5*, where we cover topics such as on-demand feature functions, feature lookups, syncing to the online store, and the Databricks Feature Engineering client.